

Nathan Krieger
HW 2
02/26/26

1. Download the Evosuite program
2. Generate test cases for both programs
3. Now run Evosuite on each program

a.

TARGET_CLASS,criterion,Coverage,Total_Goals,Covered_Goals
tutorial.Stack,LINE;BRANCH;EXCEPTION;WEAKMUTATION;OUTPUT;METHOD;METHODNOEXCEPTION;CBRANCH,0.9286971830985915,131,124
triangle.TriangleType,LINE;BRANCH;EXCEPTION;WEAKMUTATION;OUTPUT;METHOD;METHODNOEXCEPTION;CBRANCH,0.9375,173,172

b.

Triangle: 20 tests

Tutorial: 10 tests (many tests have multiple asserts)

c. ISOSCELES Example:

```
73  @Test(timeout = 4000)
74  public void test09() throws Throwable {
75      Triangle triangle0 = TriangleType.triangle(739, 1288, 1288);
76      assertEquals(Triangle.ISOSCELES, triangle0);
77  }
```

d. Stack maximizing example:

```

107     @Test(timeout = 4000)
108     public void test6() throws Throwable {
109         Integer integer0 = new Integer(12);
110         Stack<Integer> stack0 = new Stack<Integer>();
111         stack0.push(integer0);
112         stack0.push(integer0);
113         stack0.push(integer0);
114         stack0.push(integer0);
115         stack0.push(integer0);
116         stack0.push((Integer) null);
117         stack0.push(integer0);
118         stack0.push(integer0);
119         stack0.push(integer0);
120         stack0.push(integer0);
121         stack0.push((Integer) null);
122         stack0.push(integer0);
123         // Undeclared exception!
124         try {
125             stack0.push(integer0);
126             fail("Expecting exception: RuntimeException");
127
128         } catch (RuntimeException e) {
129             //
130             // Stack exceeded capacity!
131             //
132             verifyException("tutorial.Stack", e);
133         }
134     }

```

4.

Running the tests:

```
nathan-krieger > main - java org.junit.runner.JUnitCore tutorial.Stack_ESTest
JUnit version 4.13.2
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/nathan-krieger/git/SE417/HW2/evosuite-assign2-spring26/evosuite-assign2/evosuite-standalone-runtime-1.2.0.jar) to field java.awt.GraphicsEnvironment.headless
WARNING: Please consider reporting this to the maintainers of org.evosuite.runtime.GuiSupport
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
.....
Time: 0.343
OK (10 tests)

[~/git/SE417/HW2/evosuite-assign2-spring26/evosuite-assign2]
nathan-krieger > main - java org.junit.runner.JUnitCore triangle.TriangleType_ESTest
JUnit version 4.13.2
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/nathan-krieger/git/SE417/HW2/evosuite-assign2-spring26/evosuite-assign2/evosuite-standalone-runtime-1.2.0.jar) to field java.awt.GraphicsEnvironment.headless
WARNING: Please consider reporting this to the maintainers of org.evosuite.runtime.GuiSupport
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
.....
Time: 0.355
OK (20 tests)
```

a.

Running the tests again after injecting the fault:

```
nathan-krieger ➤ main - java org.junit.runner.JUnitCore triangle.TriangleType_ESTest
JUnit version 4.13.2
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/nathan-krieger/git/SE4
17/HW2/evosuite-assign2-spring26/evosuite-assign2/evosuite-standalone-runtime-1.2.0.jar) to field java.a
wt.GraphicsEnvironment.headless
WARNING: Please consider reporting this to the maintainers of org.evosuite.runtime.GuiSupport
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
.....
Time: 0.38
OK (20 tests)
```

b. No. The fault was not found.

```
79     @Test(timeout = 4000)
80     public void test10() throws Throwable {
81         Triangle triangle0 = TriangleType.triangle(2958, 2958, 2759);
82         assertEquals(Triangle.SCALENE, triangle0);
83     }
```

Since evosuite is generating tests based on the logic in the code, its not actually checking the logic itself. Here we see a triangle with $s1 = s2$ so it should be an ISOSCELES but the if-statement

```
if ((s1 == s3) || (s1<=s3) || (s2 == s3))
```

Doesn't check if $s1 == s2$ like it should, so it returns SCALENE instead.

c. Running the original tests again:

```
nathan-krieger > main -> java org.junit.runner.JUnitCore triangle.TriangleType_ESTest
JUnit version 4.13.2
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/nathan-krieger/git/SE4
17/HW2/evosuite-assign2-spring26/evosuite-assign2/evosuite-standalone-runtime-1.2.0.jar) to field java.a
wt.GraphicsEnvironment.headless
WARNING: Please consider reporting this to the maintainers of org.evosuite.runtime.GuiSupport
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
.....E...E.....
Time: 0.45
There were 2 failures:
1) test08(triangle.TriangleType_ESTest)
java.lang.AssertionError: expected:<SCALENE> but was:<ISOSCELES>
    at org.junit.Assert.fail(Assert.java:89)
    at org.junit.Assert.failNotEquals(Assert.java:835)
    at org.junit.Assert.assertEquals(Assert.java:120)
    at org.junit.Assert.assertEquals(Assert.java:146)
    at triangle.TriangleType_ESTest.test08(TriangleType_ESTest.java:70)
2) test01(triangle.TriangleType_ESTest)
java.lang.AssertionError: expected:<ISOSCELES> but was:<SCALENE>
    at org.junit.Assert.fail(Assert.java:89)
    at org.junit.Assert.failNotEquals(Assert.java:835)
    at org.junit.Assert.assertEquals(Assert.java:120)
    at org.junit.Assert.assertEquals(Assert.java:146)
    at triangle.TriangleType_ESTest.test01(TriangleType_ESTest.java:28)

FAILURES!!!
Tests run: 20, Failures: 2
```

The original test cases caught the fault.

```
@Test(timeout = 4000)
public void test08() throws Throwable {
    Triangle triangle0 = TriangleType.triangle(792, 1365, 2136);
    assertEquals(Triangle.SCALENE, triangle0);
}
```

Test 8 fails because a triangle with 3 different sides (792, 1365, and 2136) should be returning SCALENE because no 2 sides are of equal length. However, the if condition

```
if ((s1 == s3) || (s1<=s3) || (s2 == s3))
```

To check whether a triangle is ISOSCELES is met because (s1 <= s3) is satisfied with 792 <= 2136.

Another test case that caught the fault:

```
25 @Test(timeout = 4000)
26 public void test01() throws Throwable {
27     Triangle triangle0 = TriangleType.triangle(67, 67, 1);
28     assertEquals(Triangle.ISOSCELES, triangle0);
29 }
```

Test 1 fails because it should return an isosceles triangle since s1 and s2 are the same length. However, the if statement

```
if ((s1 == s3) || (s1<=s3) || (s2 == s3))
```

Is not satisfied because it is incorrectly not checking if s1 == s2 like it should. Instead, it evaluates false and returns SCALENE.

5.

a. statistics.csv:

```
HW2 > evosuite-assign2-spring26 > evosuite-assign2 > evosuite-report > statistics.csv
1 TARGET_CLASS,criterion,Coverage,Total_Goals,Covered_Goals
2 tutorial.Stack,LINE;BRANCH;EXCEPTION;WEAKMUTATION;OUTPUT;METHOD;METHODNOEXCEPTION;CBRANCH,0.9286971830985915,131,124
3 triangle.TriangleType,LINE;BRANCH;EXCEPTION;WEAKMUTATION;OUTPUT;METHOD;METHODNOEXCEPTION;CBRANCH,0.9375,173,172
4 prime.PrimeNumberFinder,LINE;BRANCH;EXCEPTION;WEAKMUTATION;OUTPUT;METHOD;METHODNOEXCEPTION;CBRANCH,0.90109940872136,323,305
5
```

b. Test file:

```
1  /*
2   * This file was automatically generated by EvoSuite
3   * Wed Feb 25 18:29:00 GMT 2026
4   */
5
6  package prime;
7
8  import org.junit.Test;
9  import static org.junit.Assert.*;
10 import static org.evosuite.runtime.EvoAssertions.*;
11 import java.util.Collection;
12 import java.util.LinkedList;
13 import java.util.List;
14 import org.evosuite.runtime.EvoRunner;
15 import org.evosuite.runtime.EvoRunnerParameters;
16 import org.junit.runner.RunWith;
17 import prime.PrimeNumberFinder;
18
19 @RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS = true, useVNET = true, resetStaticState = true,
20 public class PrimeNumberFinder_ESTest extends PrimeNumberFinder_ESTest_scaffolding {
21
22     @Test(timeout = 4000)
23     public void test00() throws Throwable {
24         List<Integer> list0 = PrimeNumberFinder.findPrimes(1676, 239);
25         assertFalse(list0.contains(239));
26     }
27
28     @Test(timeout = 4000)
29     public void test01() throws Throwable {
30         Integer integer0 = new Integer(0);
31         List<Integer> list0 = List.of(integer0, integer0, integer0, integer0);
32         int int0 = PrimeNumberFinder.computeSumOfPrimes(list0);
33         assertEquals(0, int0);
34     }
35
36     @Test(timeout = 4000)
37     public void test02() throws Throwable {
38         LinkedList<Integer> linkedList0 = new LinkedList<Integer>();
39         Integer integer0 = new Integer((-1402));
40         linkedList0.add(integer0);
41         int int0 = PrimeNumberFinder.computeSumOfPrimes(linkedList0);
42         assertEquals((-1402), int0);
43     }
44
45     @Test(timeout = 4000)
46     public void test03() throws Throwable {
47         PrimeNumberFinder.findPrimes(0, 1907);
48         // Undeclared exception!
49         PrimeNumberFinder.findPrimes((-6940), 1907);
50     }
51
52     @Test(timeout = 4000)
53     public void test04() throws Throwable {
54         // Undeclared exception!
55         try {
56             PrimeNumberFinder.computeSumOfPrimes((List<Integer>) null);
57         } catch (Exception e) {
58             // Expected exception: null
59         }
60     }
61 }
```

```

57         fail("Expecting exception: NullPointerException");
58
59     } catch(NullPointerException e) {
60         //
61         // no message in exception (getMessage() returned null)
62         //
63         verifyException("prime.PrimeNumberFinder", e);
64     }
65 }
66
67 @Test(timeout = 4000)
68 public void test05() throws Throwable {
69     LinkedList<Integer> linkedList0 = new LinkedList<Integer>();
70     // Undeclared exception!
71     try {
72         PrimeNumberFinder.computeSumOfPrimes(linkedList0);
73         fail("Expecting exception: IndexOutOfBoundsException");
74
75     } catch(IndexOutOfBoundsException e) {
76         //
77         // Index: 0, Size: 0
78         //
79         verifyException("java.util.LinkedList", e);
80     }
81 }
82
83 @Test(timeout = 4000)
84 public void test06() throws Throwable {
85     LinkedList<Integer> linkedList0 = new LinkedList<Integer>();
86     List<Integer> list0 = List.copyOf((Collection<? extends Integer>) linkedList0);
87     // Undeclared exception!
88     try {
89         PrimeNumberFinder.computeSumOfPrimes(list0);
90         fail("Expecting exception: ArrayIndexOutOfBoundsException");
91
92     } catch(ArrayIndexOutOfBoundsException e) {
93         //
94         // no message in exception (getMessage() returned null)
95         //
96     }
97 }
98
99 @Test(timeout = 4000)
100 public void test07() throws Throwable {
101     boolean boolean0 = PrimeNumberFinder.isPrime(371);
102     assertFalse(boolean0);
103 }
104
105 @Test(timeout = 4000)
106 public void test08() throws Throwable {
107     boolean boolean0 = PrimeNumberFinder.isPrime(2045);
108     assertFalse(boolean0);
109 }

```

```

110
111 @Test(timeout = 4000)
112 public void test09() throws Throwable {
113     boolean boolean0 = PrimeNumberFinder.isPrime(5);
114     assertTrue(boolean0);
115 }
116
117 @Test(timeout = 4000)
118 public void test10() throws Throwable {
119     boolean boolean0 = PrimeNumberFinder.isPrime(2073);
120     assertFalse(boolean0);
121 }
122
123 @Test(timeout = 4000)
124 public void test11() throws Throwable {
125     boolean boolean0 = PrimeNumberFinder.isPrime(1676);
126     assertFalse(boolean0);
127 }
128
129 @Test(timeout = 4000)
130 public void test12() throws Throwable {
131     boolean boolean0 = PrimeNumberFinder.isPrime(2);
132     assertTrue(boolean0);
133 }
134
135 @Test(timeout = 4000)
136 public void test13() throws Throwable {
137     boolean boolean0 = PrimeNumberFinder.isPrime(3);
138     assertTrue(boolean0);
139 }
140
141 @Test(timeout = 4000)
142 public void test14() throws Throwable {
143     boolean boolean0 = PrimeNumberFinder.isPrime(7);
144     assertTrue(boolean0);
145 }
146
147 @Test(timeout = 4000)
148 public void test15() throws Throwable {
149     boolean boolean0 = PrimeNumberFinder.isPrime((-32));
150     assertFalse(boolean0);
151 }
152
153 @Test(timeout = 4000)
154 public void test16() throws Throwable {
155     boolean boolean0 = PrimeNumberFinder.isPrime(29929);
156     assertFalse(boolean0);
157 }
158
159 @Test(timeout = 4000)
160 public void test17() throws Throwable {
161     List<Integer> list0 = PrimeNumberFinder.findPrimes(239, 239);
162     assertTrue(list0.contains(239));
163
164     int int0 = PrimeNumberFinder.computeSumOfPrimes(list0);
165     assertEquals(239, int0);
166 }
167
168 @Test(timeout = 4000)
169 public void test18() throws Throwable {
170     List<Integer> list0 = PrimeNumberFinder.findPrimes(0, 2059);
171     int int0 = PrimeNumberFinder.computeSumOfPrimes(list0);
172     assertEquals(310, list0.size());
173     assertEquals(291229, int0);
174 }
175
176 @Test(timeout = 4000)
177 public void test19() throws Throwable {
178     PrimeNumberFinder primeNumberFinder0 = new PrimeNumberFinder();
179 }
180 }
181

```

c. Test results:


```

nathan-krieger main -> java -cp ".:Prime:lib/junit-4.13.2.jar:lib/hamcrest-core-1.3
.jar:evosuite-standalone-runtime-1.2.0.jar" org.junit.runner.JUnitCore prime.PrimeNumberF
inder_ESTest
JUnit version 4.13.2
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/nathan-
krieger/git/SE417/HW2/evosuite-assign2-spring26/evosuite-assign2/evosuite-standalone-runt
ime-1.2.0.jar) to field java.awt.GraphicsEnvironment.headless
WARNING: Please consider reporting this to the maintainers of org.evosuite.runtime.GuiSup
port
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective acces
s operations
WARNING: All illegal access operations will be denied in a future release
.....
Time: 0.38

OK (20 tests)

```

- d. There are 2 test cases that surprise me.

First,

```

35
36     @Test(timeout = 4000)
37     public void test02() throws Throwable {
38         LinkedList<Integer> linkedList0 = new LinkedList<Integer>();
39         Integer integer0 = new Integer((-1402));
40         linkedList0.add(integer0);
41         int int0 = PrimeNumberFinder.computeSumOfPrimes(linkedList0);
42         assertEquals((-1402), int0);
43     }
44

```

This test surprises me because it's smart enough to check negative numbers. I would have only thought to check positive numbers but this shows an interesting edge case with primes.

The oracle states that `computeSumOfPrimes([-1402])` must return -1402. This reveals that the method is not checking if the numbers are prime and reveals a design assumption.

Second,

```
52  @Test(timeout = 4000)
53  public void test04() throws Throwable {
54      // Undeclared exception!
55      try {
56          PrimeNumberFinder.computeSumOfPrimes((List<Integer>) null);
57          fail("Expecting exception: NullPointerException");
58
59      } catch(NullPointerException e) {
60          //
61          // no message in exception (getMessage() returned null)
62          //
63          verifyException("prime.PrimeNumberFinder", e);
64      }
65  }
66  }
```

This test passes in a null list to compute the sum of. This is something I personally would not have thought of when writing tests.

The oracle is that the method must throw a NullPointerException. The test passes only if this exception is thrown, making the expected exception the oracle for this test.

6.

- a. Coverage report for automatically generated tests in jacoco ran on the PrimeNumberFinder.java from Canvas (*the HW assignment instructions are unclear if I should run it on that or my own PrimeNumberFinder.java from HW1*):

PrimeNumberFinder

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
isPrime(int)		100%		95%	1	13	0	12	0	1
computeSumOfPrimes(List)		100%		100%	0	3	0	7	0	1
findPrimes(int, int)		100%		100%	0	3	0	5	0	1
PrimeNumberFinder()		100%		n/a	0	1	0	1	0	1
Total	0 of 117	100%	1 of 32	96%	1	20	0	25	0	4

PrimeNumberFinder.java

```
1.  /* program created by ChatGPT bsaed on the prime program in
2.  our textbook, with prompts from M.Cohen and then modified by adding (both)
3.  correct and faulty code by M. Cohen 2025. No guarantee of correctness of program or
4.  formal specifications were provided. Specifications for purposes of testing and
5.  expected behavior will be given on the assignment handout. */
6.  //package prime;
7.
8.  import java.util.ArrayList;
9.  import java.util.List;
10.
11. public class PrimeNumberFinder {
12.
13.     /* you can uncomment out the main method and run the program from command line if you like.
14.     For running the test and coverage you should comment this back out */
15.
16.     /* public static void main(String[] args) {
17.     if (args.length != 2) {
18.         System.out.println("Usage: java PrimeNumberFinder <lowerBound> <upperBound>");
19.         return;
20.     }
21.
22.     int lowerBound = Integer.parseInt(args[0]);
23.     int upperBound = Integer.parseInt(args[1]);
24.     int sumofP=0;
25.
26.     List<Integer> primes = findPrimes(lowerBound, upperBound);
27.     sumofP=computeSumOfPrimes(primes);
28.     System.out.println("Prime numbers between " + lowerBound + " and " + upperBound + ": " + primes);
29.     System.out.println("Sum of prime numbers between " + lowerBound + " and " + upperBound + ": " + sumofP);
30. }*/
31.
32.
33.
34.     /* method to find primes between (including) two numbers */
35.     public static List<Integer> findPrimes(int lowerBound, int upperBound) {
36.         List<Integer> primeNumbers = new ArrayList<>();
37.
38.         for (int number = lowerBound; number <= upperBound; number++) {
39.             if (isPrime(number)) {
40.                 primeNumbers.add(number);
41.             }
42.         }
43.
44.         return primeNumbers;
45.     }
46.
47.     /* method to compute the sum of primes given a list of prime numbers */
48.
49.     public static int computeSumOfPrimes(List<Integer> primes) {
50.         int sum = 0;
51.
52.         if(primes.size()>1){
53.             for (int prime : primes) {
54.                 sum += prime;
55.             }
56.         }
57.         else
58.             sum=primes.get(0);
59.         return sum;
60.     }
61.
62.
63.     /* method to ask if a single number is prime */
64.
65.
66.     public static boolean isPrime(int num) {
67.         if (num <= 1) {
68.             return false;
69.         }
70.
71.         if (num == 2 || num == 3 || num == 7) {
72.             return true;
73.         }
```

```

64.    /* method to ask if a single number is prime */
65.
66.
67.    public static boolean isPrime(int num) {
68.    ◆ if (num <= 1) {
69.        return false;
70.    }
71.
72.    ◆ if (num == 2 || num == 3 || num == 7) {
73.        return true;
74.    }
75.
76.    ◆ if (num % 2 == 0 || num % 3 == 0 || num % 6 == 0) {
77.        return false;
78.    }
79.
80.    ◆ if (num > 5 && num % 5 == 0) {
81.        return false;
82.    }
83.
84.    ◆ for (int i = 5; i * i <= num; i += 6) {
85.    ◆     if (num % i == 0 || num % (i + 2) == 0) {
86.        return false;
87.    }
88.    }
89.
90.    return true;
91.    }
92.
93.
94.    }

```

The evosuite automatically generated tests were able to get 100% code coverage because it creates tests based on the code itself and works backwards. It was not able to achieve 100% branch coverage because the PrimeNumberFinder.java code given to us still has the extra condition (num % 6 == 0) that was resolved by the end of HW1.

- b. The console when I ran the auto generated tests on the code.

```

[INFO]
[INFO] --- maven-surefire-plugin:2.22.0:test (default-test) @ org.jacoco.examples.maven ---
[INFO]
[INFO] -----
[INFO]  T E S T S
[INFO] -----
[INFO] Running PrimeNumberFinderTest
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.051 s - in PrimeNumberFinderTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.295 s
[INFO] Finished at: 2026-02-25T15:15:31-06:00
[INFO] -----

```

As seen above, all the tests pass, even though we know that there is full coverage, and there is an exception that should be getting thrown in the computeSumOfPrimes() method. There are 2 tests that should have caught this:

```

@Test(timeout = 4000)
public void test05() throws Throwable {
    LinkedList<Integer> linkedList0 = new LinkedList<Integer>();
    // Undeclared exception!
    assertThrows(IndexOutOfBoundsException.class, () -> {
        PrimeNumberFinder.computeSumOfPrimes(linkedList0);
    });
}

@Test(timeout = 4000)
public void test06() throws Throwable {
    LinkedList<Integer> linkedList0 = new LinkedList<Integer>();
    List<Integer> list0 = List.copyOf((Collection<? extends
Integer>) linkedList0);
    // Undeclared exception!
    assertThrows(ArrayIndexOutOfBoundsException.class, () -> {
        PrimeNumberFinder.computeSumOfPrimes(list0);
    });
}

```

However, since evosuite generates tests that pass based on the code, it created tests that insisted that the error was thrown instead of expecting there to be some error handling in the method itself. This highlights the dangers of automatically generating tests like this. Overall I think evosuite is a cool program, but should be used carefully. I see strong potential for usefulness with regression testing, but not so much when creating new code and testing it.